CSE 144 Final Project Report

Transfer Learning Challenge

Group Members:

Michael Pimentel (CruzID: 1993245)

Tanisha Prabhu (Cruz ID: 2164376)

Spring 2026

1 Introduction

1. Problem goal and setting. The goal is to build a 100-class image classifier using transfer learning, submitted to the UCSC CSE 144 Spring 2026 Kaggle competition. Given a small, fixed training set of 1,000 labeled images across 100 classes (10 per class), the model must generalize to 1,036 unlabeled test images and maximize top-1 classification accuracy on the public leaderboard.

2. Why transfer learning is appropriate. With only ~10 labeled images per class, training a deep network from scratch would severely overfit. Transfer learning from EfficientNet-V2-M, pretrained on ImageNet-1K with ~1.2M images across 1,000 classes — provides rich, general-purpose visual features that dramatically reduce the data required to learn discriminative boundaries across all 100 classes.

3. Brief summary of approach and main result. EfficientNet-V2-M was fine-tuned in two phases: first only the new classification head was trained for 10 epochs (backbone frozen), then all layers were jointly fine-tuned for 80 more epochs at a lower learning rate. Test-Time Augmentation (TTA) with 4 flip variants was applied at inference. The model achieved a Kaggle public leaderboard score of 0.80909 (Rank #33).

2 Dataset

1. Number of classes and dataset sizes (train/val/test). The dataset has 100 classes with approximately 10 images per class (~1,000 images total in the training folder). After an 80/20 stratified split:

- Train: ~800 images, 80% stratified split, seed=42
- Validation: ~200 images, 20% stratified split, seed=42
- Test: 1,036 images (unlabeled)

2. Directory structure and label mapping details. Training data is organized as train/<class_id>/ where class_id ranges from 0 to 99. A custom NumericalImageFolder class sorts directories numerically rather than alphabetically, ensuring that folder "2" maps to label 2 and not a lexicographic index. Test images are named 0.jpg through 1035.jpg.

Directory structure:

final_project/

```
├── train/
│   ├── 0/          # ~10 images per class
│   ├── 1/
│   └── ...         # 100 classes total (folders named 0–99)
└── test/
        ├── 0.jpg
        ├── 1.jpg
        └── ...     # 1,036 images
```

3. Preprocessing and data augmentation. All images are resized to 384×384 and normalized with ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).

Augmentation	Parameters	Applied To
RandomHorizontalFlip	p=0.5	Train only
RandomVerticalFlip	p=0.5	Train only
RandomRotation	±15°	Train only
ColorJitter	brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1	Train only
RandomErasing	p=0.25	Train only
Resize + Normalize	384×384, ImageNet stats	Train + Val + Test

Note: MixUp (α=0.4) was tested but disabled. It over-regularized when combined with label smoothing and RandomErasing, hurting validation accuracy.

3 Implementation

3.1 Model

1. Pretrained backbone. EfficientNet-V2-M (torchvision, IMAGENET1K_V1 weights) was selected for its strong accuracy-to-parameter ratio and native 384×384 input resolution. It

achieves ~85.1% top-1 accuracy on ImageNet and has 54M parameters — large enough to extract rich features without exceeding GPU memory constraints at batch size 16.

2. Architecture changes. The original ImageNet classifier (a single 1000-class linear layer) was replaced with:

EfficientNet-V2-M backbone (feature extractor, 1280-dim output) └── Dropout(p=0.3) └── Linear(1280 → 100)

The Dropout layer adds regularization before the final class projection to reduce overfitting on the small per-class sample count.

3. Fine-tuning strategy. A two-phase strategy was used:

- Phase 1 — Head only (10 epochs): Backbone frozen, only the new classifier is trained at LR=1e-3. This prevents the randomly initialized head weights from corrupting the pretrained backbone during early training.
- Phase 2 — Full fine-tune (80 epochs): All layers unfrozen, jointly trained at a lower LR=1e-4. The best checkpoint by validation accuracy is saved throughout both phases.

3.2 Training

1. Loss function and optimizer.

- Loss: CrossEntropyLoss with label smoothing ε=0.1 (softens one-hot targets to reduce overconfidence)
- Optimizer: AdamW with weight decay=1e-4

2. Hyperparameters.

Parameter	**Phase 1**	**Phase 2**
Epochs	10	80
Learning Rate	1e-3	1e-4
LR Scheduler	CosineAnnealingLR (T_max=10)	CosineAnnealingLR (T_max=80)
Batch Size	16	16
Weight Decay	1e-4	1e-4
Image Size	384×384	384×384

3. Hardware/software environment.

- Platform: Kaggle Notebook (GPU — T4/P100)
- Python packages: torch>=2.0.0, torchvision>=0.15.0, scikit-learn>=1.2.0, Pillow>=9.0.0, numpy>=1.24.0
- Device selection: CUDA → MPS (Apple Silicon) → CPU fallback

4 Experiments

1. Baseline setup. Baseline: Phase 1 only — frozen EfficientNet-V2-M backbone, linear head trained for 10 epochs at LR=1e-3. This establishes how much the pretrained features alone can achieve without any backbone fine-tuning.

2. Hyperparameter tuning plan and validation method. All decisions used the 80/20 stratified validation split (same seed=42) as the evaluation signal. Key axes tuned:

- Image resolution: 224px vs. 384px
- LR schedule: Step decay vs. CosineAnnealingLR
- Label smoothing: 0.0 vs. 0.1
- Augmentation combinations (see ablations below)

3. Ablations.

Component	**Variant Tested**	**Result**
MixUp	α=0.4 vs. disabled	Disabled, hurts when combined with label smoothing + RandomErasing
RandomErasing	p=0 vs. p=0.25	p=0.25 improves generalization
Image size	224px vs. 384px	384px improves accuracy; higher memory cost
Freezing strategy	End-to-end vs. 2-phase	2-phase outperforms; avoids head destroying backbone features early
Label smoothing	ε=0.0 vs. ε=0.1	ε=0.1 reduces overconfidence

5 Results

1. Training/validation accuracy. Best validation accuracy: 82.87%. Final training accuracy: 100% (overfitting)

2. Kaggle public leaderboard score. 0.80909 (Rank #33 — best entry, improved from 0.50909)

Submission was generated by inference.py using 4-pass TTA (original, horizontal flip, vertical flip, both flips). Softmax probabilities are averaged across all passes before taking argmax.

6 Discussion

1. What worked best and why. The two-phase fine-tuning strategy was the single most impactful design decision. Training only the randomly-initialized head first (frozen backbone) prevents early gradient updates from the random head from destroying the pretrained backbone features. Once the head has reasonable weights, jointly fine-tuning all layers at a 10× lower learning rate yields significant accuracy gains. CosineAnnealingLR provided smooth convergence without requiring manual learning rate tuning. TTA with flip variants added a cost-free inference boost.

2. Failure cases, overfitting/underfitting observations. With only ~8 training images per class (after the val split), overfitting is the primary risk. Label smoothing, RandomErasing, ColorJitter, and the two-phase LR strategy collectively reduced the train/val gap. Despite this, some overfitting was still observed in later Phase 2 epochs, motivating the best-checkpoint strategy rather than using the final epoch weights. Visually similar classes are expected to contribute the most confusion in error analysis.

3. Limitations and concrete next improvements. Limitation: The dataset is extremely small (~8 images/class for training), making generalization inherently difficult. Improvements:

1. Larger backbone (EfficientNet-V2-L or ViT-L/16) for stronger pretrained features
2. More aggressive augmentation: AugMix, RandAugment, or CutMix
3. Ensemble of multiple checkpoints (different seeds or backbones)
4. Pseudo-labeling: use high-confidence test predictions to augment training data
5. Longer Phase 2 fine-tuning with early stopping on validation loss

7 Reproducibility

1. Random seeds and determinism settings.

SEED = 42 random.seed(SEED) np.random.seed(SEED) torch.manual_seed(SEED) torch.cuda.manual_seed_all(SEED) torch.backends.cudnn.deterministic = True torch.backends.cudnn.benchmark = False

The stratified train/val split also uses random_state=SEED.

2. Package versions / environment setup steps.

python3 -m venv venv source venv/bin/activate # Windows: venv\Scripts\activate pip install -r requirements.txt

requirements.txt: torch>=2.0.0 torchvision>=0.15.0 scikit-learn>=1.2.0 Pillow>=9.0.0 kagglehub==0.3.12 numpy>=1.24.0 matplotlib>=3.7.0

3. Exact commands to train and to generate submission.csv.

Step 1: Train (saves best_model.pth)

python train.py

Step 2: Generate submission CSV with TTA

python inference.py

Both scripts run on Kaggle Notebooks with dataset paths:

- Train: /kaggle/input/competitions/ucsc-cse-144-spring-2026-final-project/train
- Test: /kaggle/input/competitions/ucsc-cse-144-spring-2026-final-project/test
- Checkpoint: /kaggle/working/best_model.pth
- Output: /kaggle/working/submission.csv

8 Team Contributions

1. Michael Pimentel (mjpiment): Dataset inspection and label mapping fix (NumericalImageFolder) and model architecture design.
2. Tanisha Prabhu (taprabhu): Designed two-phase training pipeline (train.py), TTA inference pipeline (inference.py), hyperparameter tuning, and Kaggle submission.

9 References

1. Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller Models and Faster Training. ICML 2021. arXiv:2104.00298.
2. PyTorch TorchVision: Pretrained Models: https://pytorch.org/vision/stable/models.html